Rheinische Friedrich-Wilhelms-Universität Bonn

Bachelorstudiengang Informatik

Prüfungsklausur

Semester: WS ☑ SS ☐ *2013/14*	Versuch: 1. ☐ 2. ☑
Modulbezeichnung: ***Systemnahe Programmierung***	
Modulnummer: *BA-INF 034/129*	Prüfungsdatum: 27. *März 2014*
1. Prüfer: *Prof. Dr. Michael Meier*	2. Prüfer: *Dr. Matthias Frank*

vom Prüfling auszufüllen:

Name:	Matrikel-Nr:
Vorname:	Semester:
Studiengang Bachelor Informatik ☐ Lehramt ☐ Informatik als Nebenfach ☐	

vom Prüfer auszufüllen:

Bewertung:

*Noten und Notenwerte: **sehr gut (1,0; 1,3) – gut (1,7; 2,0; 2,3) – befriedigend (2,7; 3,0; 3,3) – ausreichend (3,7; 4,0) – nicht ausreichend (5,0)**

Datum: 27. *März 2014*

---------- ----------

Unterschrift des 1. Prüfers Unterschrift des 2. Prüfers

Prof. Dr. Michael Meier
Dr. Matthias Frank

Bonn, den 27. März 2014

Hinweise zur Modulprüfung zu „Systemnahe Programmierung“ (BA-INF 034/129)

Wintersemester 2013/2014
9:30 Uhr – 11:00 Uhr

Hinweise (GENAU DURCHLESEN!)

- Neben Papier und Schreibutensilien sind **keine weiteren Hilfsmittel** erlaubt. Verwenden Sie nur dokumentenechte Stifte (z.B. keine Bleistifte). Verwenden Sie keine roten Stifte.
- Vergessen Sie nicht, Ihren Namen und die Matrikelnummer auf *jedes* Blatt zu schreiben. Blätter ohne diese Angaben werden nicht gewertet.
- Schreiben Sie Ihre Lösungen auf die Aufgabenblätter möglichst in die dafür vorgesehenen Felder. Sie können auch die Rückseiten verwenden. Weiteres Schreibpapier kann von den Betreuern angefordert werden. Benutzen Sie kein mitgebrachtes Papier.
- Bitte schreiben Sie in Ihrem eigenen Interesse deutlich. Für nicht lesbare Lösungen können wir keine Punkte vergeben.
- Klausurblätter dürfen nicht voneinander getrennt werden.
- Werden mehrere unterschiedliche Lösungen für eine Aufgabe abgegeben, so wird die Aufgabe nicht gewertet.
- Im Fall von Täuschungsversuchen wird die Klausur sofort mit 0 Punkten bewertet. Eine Vorwarnung erfolgt nicht.
- In der Klausur können Sie 90 Punkte erhalten. Achten Sie darauf, dass Sie im Schnitt nicht viel mehr als 1 Minute pro Punkt zur Verfügung haben. Mit 50% dieser Gesamtpunktzahl haben Sie die Klausur sicher bestanden.

Viel Erfolg!

1	**2**	**3**	**4**	**5**	**6**	**7**	**Gesamt**
12	14	13	10	15	12	14	90

Aufgabe 1 : (Multiple-Choice, 12 Punkte)

Bei den folgenden Multiple-Choice-Fragen wird für jedes richtig gesetzte Kreuz 1 Punkt vergeben. Bei falschen Antworten werden 0 Punkte vergeben.

	Aussage	Richtig	Falsch
1	Das Netzwerkprotkoll IP (Internet Protocol) bietet zwar keine Korrektur fehlerhafter Pakete, Bitfehler in den Nutzdaten können aber erkannt werden.	☐	☐
2	Bei Verwendung der Protokollfamilie AF_INET bestimmt der Sockettyp SOCK_STREAM oder SOCK_RAW, ob TCP oder UDP als Transportprotokoll verwendet wird.	☐	☐
3	Beim Preforked Server (mit TCP) erzeugt der Haupt-Prozess zunächst eine feste Anzahl an Child-Prozessen zur parallelen Bearbeitung eingehender TCP-Verbindungen. Danach wird der Haupt-Prozess beendet.	☐	☐
4	Ein UDP-Server verwendet die Funktion listen(), um eingehende Verbindungsanfragen annehmen zu können.	☐	☐
5	Die sog. „Host-Byte-Order“ ist die eindeutige Darstellung von aus mehreren Bytes bestehenden Integer-Zahlen, die unabhängig von der konkreten Host-Architektur eingesetzt werden kann.	☐	☐
6	Kommunizieren zwei Rechner mit derselben Byte-Order über ein Cross-Link-Kabel, darf nicht auf die Network-Byte-Order-Umwandlung verzichtet werden.	☐	☐
7	Für einen nichtblockierenden Aufruf von select() muss die Systemfunktion fcntl() (File Control) eingesetzt werden.	☐	☐
8	Da x86-Assembler keine direkte Adressierung kennt, muss man zum Auslesen des Inhalts einer Adresse diese immer erst unmittelbar in ein Register laden und dann mit Register-indirekter Adressierung darauf zugreifen.	☐	☐
9	Als Prozess bezeichnet man den Programmcode eines laufenden Programms ohne dessen benutzten Heap-Speicher.	☐	☐
10	Threads teilen sich unter anderem denselben Adressraum, haben aber jeweils einen eigenen Stack.	☐	☐
11	In der „Big-Endian-Byte-Order“ wird das niederwertigste Byte einer aus mehreren Bytes bestehenden Integer-Zahl an der höchsten Adresse im Speicher abgelegt.	☐	☐
12	UDP ist zwar verbindungslos, aber dafür zuverlässig.	☐	☐

Sollten Sie der Meinung sein, dass eine (oder mehrere) der obigen Fragen missverständlich mit „Richtig“ oder „Falsch“ zu beurteilen sind, dann können Sie unten Ihre gewählte Antwort kurz begründen. Bitte achten Sie dabei selbst auf Ihre Zeiteinteilung: Es macht keinen Sinn, viele (oder gar alle) der Multiple-Choice-Fragen durch ausführlichen Text zu begründen. Es gibt nur für die oben gewählten Antworten Punkte, nicht für Ihre Erläuterungen.

Nr.	Ihre Erläuterung

Aufgabe 2: (Assembler – Programmablauf analysieren, 14 Punkte [3+3+2+4+2])

Gegeben ist das im Folgenden dargestellte Assembler-Programm. Aus Gründen der Übersichtlichkeit ist das Programm auf diese und die folgenden Seiten verteilt.

Hauptroutine des Assemblerprogramms:

```
.text

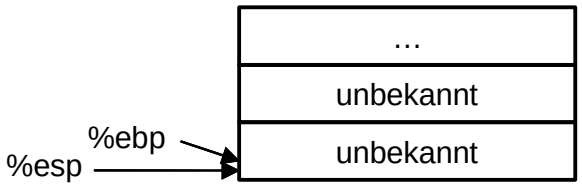
.global _start

_start:

  # call funktion(3, 9)
  pushl  $9
  pushl  $3
  call   funktion
  # Parameter wieder vom Stack nehmen
  addl   $8,%esp

  # exit process
  movl   $1,%eax
  movl   $0,%ebx
  int    $0x80
```

Stack-Aussehen nach Label "_start"



a) Beim Eintritt in die Hauptroutine des Assemblerprogramms hat der Stack das oben gezeigte Aussehen (Zeitpunkt vor der Ausführung des ersten Befehls nach dem Label „_start").

Ergänzen Sie nun den Stack (Belegung der Zellen mit Konstanten bzw. kurze Erklärung des Inhaltes), wie er nach dem Betreten der Unterroutine „funktion" (siehe übernächste Seite) beim Erreichen des Labels „Zeitpunkt_1" aussieht. Kennzeichnen Sie auch die dann aktuelle Position des Basepointers %ebp und des Stackpointers %esp.

Platz für Ihre Lösung:
(bitte streichen Sie eine ungültige Lösung deutlich)

…
unbekannt
unbekannt

Reserve-Platz für Ihre Lösung

…
unbekannt
unbekannt

b) Betrachten Sie nun die Unterroutine „funktion“ auf der nächsten Seite genauer und verfolgen Sie den weiteren Ablauf.

Ergänzen Sie jetzt den Stack (Belegung der Zellen mit Konstanten bzw. kurze Erklärung des Inhaltes), wie er nach dem Erreichen des Labels „Zeitpunkt_2“ aussieht. Kennzeichnen Sie auch hier wieder die aktuelle Position des Basepointers %ebp und des Stackpointers %esp.

Ergänzen Sie Ihre Lösung an der dafür vorgesehenen Stelle auf der nächsten Seite.

c) Welcher (fatale) Fehler wird von der Unterroutine hervorgerufen?

Lösung:

d) Verändern Sie das gegebene Programm in der Weise, dass der Fehler aus c) nicht mehr auftritt und dass das Programm das Ergebnis der letzten arithmetischen Operation gemäß der in der Vorlesung besprochenen „cdecl Calling Convention“ an den Aufrufer als Rückgabewert zurückgibt.

Lösung:
(Links: ursprüngliche Sourcezeile Rechts: veränderte Sourcezeile)

_____	_____
_____	_____
_____	_____

e) Ersetzen Sie den Befehl „leave“ in den folgenden Zeilen durch eine äquivalente Folge von Assemblerbefehlen, wobei eine beliebige Auswahl aus den folgenden Befehlen erlaubt ist: {movl, pushl, popl, addl, subl, incl, decl}

Lösung:

Unterroutine „funktion“ des Assemblerprogramms:

Platz für Ihre Lösung Teil b)

```
# int funktion(int a, b)
funktion:

  # set up stack frame
  pushl %ebp
  movl  %esp,%ebp

Zeitpunkt_1:

  # reserviere Platz für 2 lokale Variablen
  subl  $8,%esp


  movl  8(%ebp), %eax
  movl  %eax, -4(%ebp)

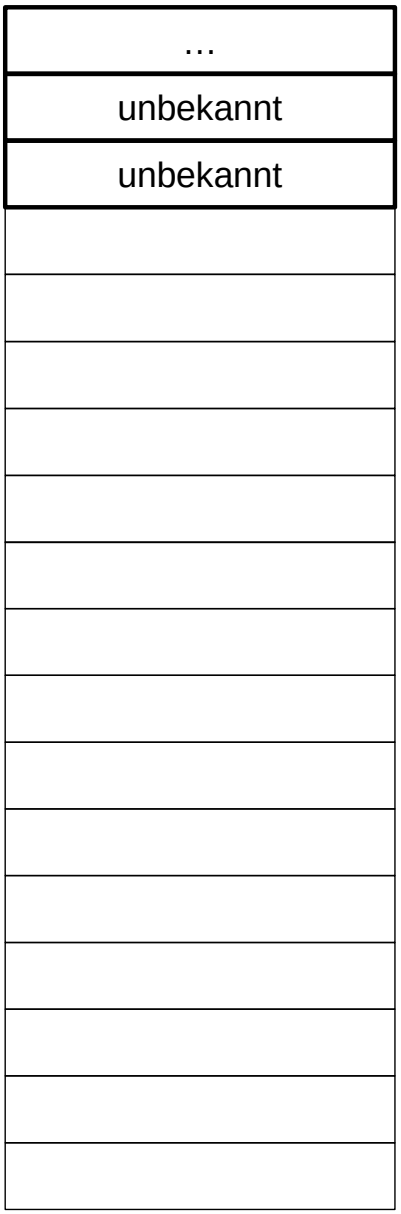
  movl  12(%ebp), %ebx
  movl  %ebx, -8(%ebp)

  addl  %eax, %ebx
  movl  %ebx, 4(%ebp)


Zeitpunkt_2:

  # reconstruct stack- and base-pointer
  leave

  # return
  ret
```



Reserve-Platz auf der folgenden Seite.
(bitte streichen Sie eine ungültige Lösung deutlich)

Unterroutine „funktion“ des Assemblerprogramms:

Reserve-Platz für Ihre Lösung Teil b)
(bitte streichen Sie eine ungültige Lösung deutlich)

```
# int funktion(int a, b)
funktion:

  # set up stack frame
  pushl %ebp
  movl  %esp,%ebp

Zeitpunkt_1:

  # reserviere Platz für 2 lokale Variablen
  subl  $8,%esp


  movl  8(%ebp), %eax
  movl  %eax, -4(%ebp)

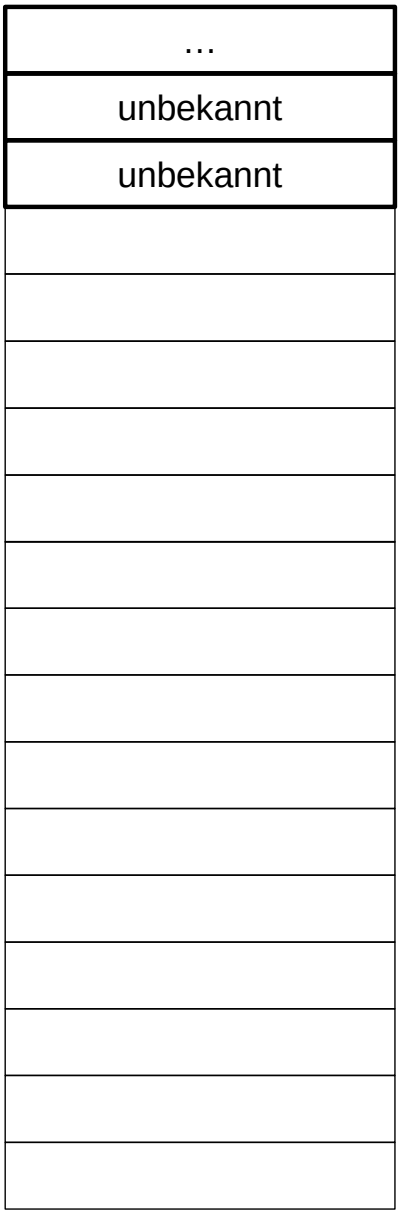
  movl  12(%ebp), %ebx
  movl  %ebx, -8(%ebp)

  addl  %eax, %ebx
  movl  %ebx, 4(%ebp)


Zeitpunkt_2:

  # reconstruct stack- and base-pointer
  leave

  # return
  ret
```



Aufgabe 3: (Busy Waiting in Threads, 13 Punkte [1 + 12])

Gegeben sei das folgende Programmfragment, in dem ein Thread in der Funktion „waitingThread“ so lange per Busy Waiting wartet, bis ein anderer Thread mit Hilfe der Funktion „setFlag“ einen Flagwert größer als Null setzt.

Hinweis: Am Ende der Klausur finden Sie eine Liste mit der Syntax relevanter Bibliotheksfunktionen.

```
int threadFlag = 0;
pthread_mutex_t threadFlagMutex;

void* waitingThread (void* thread_arg) {
     int localFlag = 0;
     while (!localFlag) {
          pthread_mutex_lock(&threadFlagMutex);
          localFlag = threadFlag;
          pthread_mutex_unlock(&threadFlagMutex);
     }
     printf("Condition fulfilled!\n");
     return 0;
}

void setFlag (int newFlagValue) {
     pthread_mutex_lock(&threadFlagMutex);
     threadFlag = newFlagValue;
     pthread_mutex_unlock(&threadFlagMutex);
}
```

a) (1 Punkt) Was ist der Hauptnachteil bei der Verwendung von Busy Waiting?

b) (12 Punkte) Ändern Sie die beiden Funktionen „waitingThread“ und „setFlag“ so ab, dass kein Busy Waiting mehr verwendet wird, sondern der in der Vorlesung diskutierte alternative Ansatz. Die Semantik des Programms soll ansonsten unverändert bleiben.

```
int threadFlag = 0;



void* waitingThread (void* thread_arg) {









      printf("Condition fulfilled!\n");
      return 0;
}

void setFlag (int newFlagValue) {





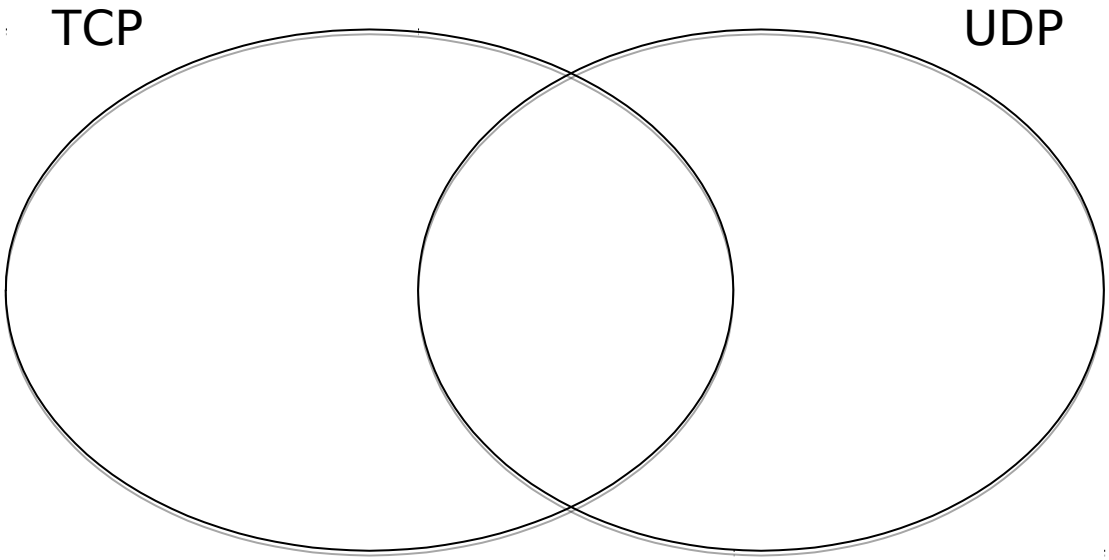




}
```

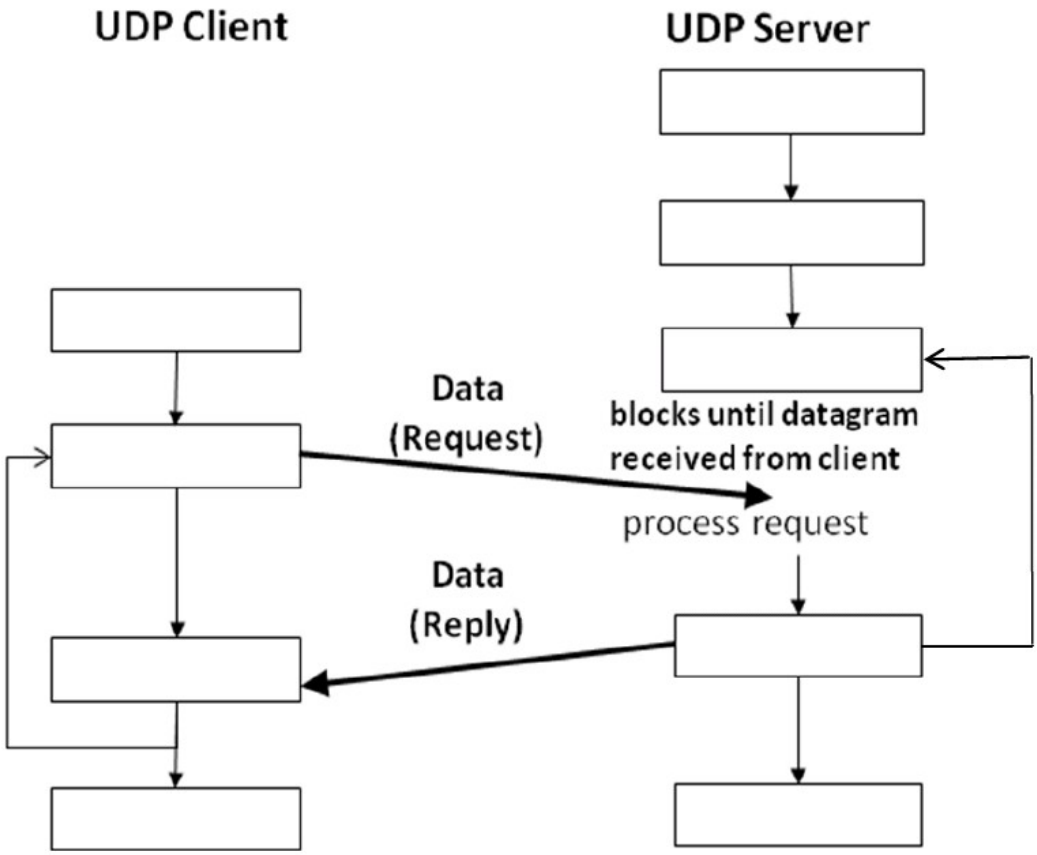
Aufgabe 4: (Sockets in C, 10 Punkte [11x0,5+9x0,5])

a) (5,5 Punkte) Ordnen Sie die folgenden Funktionsnamen zum Umgang mit Sockets den Protokollen UDP und TCP zu, indem Sie sie in den jeweiligen Mengenkreis eintragen. Ordnen Sie die Funktionen, die für beide Protokolle Verwendung finden, dem Schnittbereich der beiden Mengen zu.



Die zuzuordnenden Funktionsnamen sind: `accept()`, `bind()`, `close()`, `connect()`, `listen()`, `read()`, `recvfrom()`, `sendto()`, `shutdown()`, `socket()`, `write()`.

b) (4,5 Punkte) Ergänzen Sie das angegebene allgemeine Ablaufschema einer UDP Client/Server Kommunikation über Sockets mit den entsprechenden C-Funktionsaufrufen. Vernachlässigen Sie dabei die Parameter der Funktionen.



Aufgabe 5: (I/O-Multiplexing I/O , 15 Punkte [5 + 3 + 7])

In der Vorlesung wurden verschiedene Arten des I/O-Multiplexings vorgestellt.

a) (5 Punkte) Bitte zählen Sie alle in der Vorlesung genannten Arten auf.

1. _____
2. _____
3. _____
4. _____
5. _____

b) (3 Punkte) In der Vorlesung haben Sie das Verfahren eines „Iterativen TCP-Servers“ kennen gelernt. Der Vorteil ist die einfache Programmierbarkeit. Es gibt aber einige Nachteile dieses Ansatzes. Beschreiben Sie im folgenden Feld kurz die (drei, ggf. vier) Nachteile eines iterativen TCP-Servers.

```
Nachteile eines iterativen TCP-Servers:
```

c) (7 Punkte) Aus der Vorlesung und den Übungen ist der folgende iterative TCP-Server (Bank-Server) bekannt. Überführen Sie diesen in eine nicht blockierende Variante, die parallel mehrere Verbindungen verarbeiten kann. Nutzen Sie dafür die Systemfunktion `fork()`.

Für Ihre Lösungen können Sie sich direkt auf die Zeilennummern des Originals beziehen und müssen unveränderte Zeilen nicht abschreiben. Achten Sie jedoch auf eine klare und eindeutige Darstellung ihrer Lösung.

Hinweis: Der Server ist nur in Auszügen abgedruckt, soweit für diese Aufgabe relevant. Gehen Sie also davon aus, dass er syntaktisch korrekt ist und nicht deklarierte / initialisierte Variablen, wie von Ihnen benötigt, genutzt werden können. Ferner können Sie davon ausgehen, dass es zu keinen Fehlern während der Programmlaufzeit kommt.

```
1  int main( int argc, char **argv )
```

```
2  {
3      // Variablendeklaration erfolgt global
4      create_socket()
5      bind_socket()
6      listen_socket()
7      finish = 0;
8
9      while ( finish==0 )
10     {
11         newsock = accept( serversocket, (struct sockaddr *) &client_addr, &addrlen );
12
13         closed = 0;
14         while ( closed==0 )
15         {
16             printf ( "\nWriting amount in bank account : %i\n", bankAccount );
17             writeInt ( newsock, bankAccount );
18             printf ( "Waiting for instruction \n" );
19             inst = readInt ( newsock );
20             if ( successOfReadInt != 0 ) {
21                 closed = 1;
22                 successOfReadInt = 0;
23             } else {
24
25                 switch ( inst ) {
26                 case 1: // Einzahlen
27                     printf ( "Waiting for amount\n" );
28                     amount = readInt ( newsock );
29                     bankAccount += amount;
30                     break;
31                 case 2: // Abheben
32                     printf ( "Waiting for amount\n" );
33                     amount = readInt ( newsock );
34                     bankAccount -= amount;
35                     break;
36                 case 0: // Beenden
37                     printf ( "Disconnect from client !!!\n" );
38                     closed = 1;
39                     finish = 1;
40                 }
41             }
42         }
43
44         printf ( "Disconnecting client ...\n" );
45         err = close ( newsock );
46     }
47
48     err = close ( serversocket );
49     return 0;
50 }
```

Platz für Ihre Lösung:

Aufgabe 6: (Interprozesskommunikation, Pipes Files, 12 Punkte [6 + 2 + 4])

a) (6 Punkte) In der Vorlesung wurden diverse Konzepte und Mechanismen zur Interprozesskommunikation (Englisch IPC, Interprocess Communication) besprochen.

Erklären Sie jeweils **kurz** für jedes der unten genannten Persistenz Schemen die charakteristischen Eigenschaften, und dabei insbesondere, wie lange ein IPC-Objekt im jeweiligen Schema existieren kann:

1. Prozess-Persistenz (Englisch Process Persistence)

_____

_____

2. Kernel-Persistenz

_____

_____

3. Dateisystem-Persistenz (Englisch File System Persistence)

_____

_____

b) (2 Punkte) Welches Persistenz-Schema weisen die folgenden IPC-Mechanismen auf (ggf. kurze Erläuterung mit angeben):

1. Pipes

_____

2. FIFOs (named Pipes)

_____

3. Files

_____

4. Sockets

_____

!NICHT ZU BEARBEITEN – nur zur Orientierung!

5. Message Queues

Kernel Persistence

_____

c) (4 Punkte) Nennen Sie, wie ein IPC-Objekt des entsprechenden IPC-Konzeptes von einem Prozess zur Kommunikation eindeutig identifiziert werden kann.

Ggf. helfen Ihnen die Signaturen der entsprechenden Systemfunktionen, siehe die Liste der Syntax relevanter Bibliotheksfunktionen am Ende der Klausur.

1. Pipes

_____

2. FIFOs (named Pipes)

_____

3. Files

_____

4. Sockets (Internet Domain)

_____

!NICHT ZU BEARBEITEN – nur zur Orientierung!

5. Sockets (Unix Domain)

Identifikation über Namen, Zugriff über Socket-/File-Deskriptor

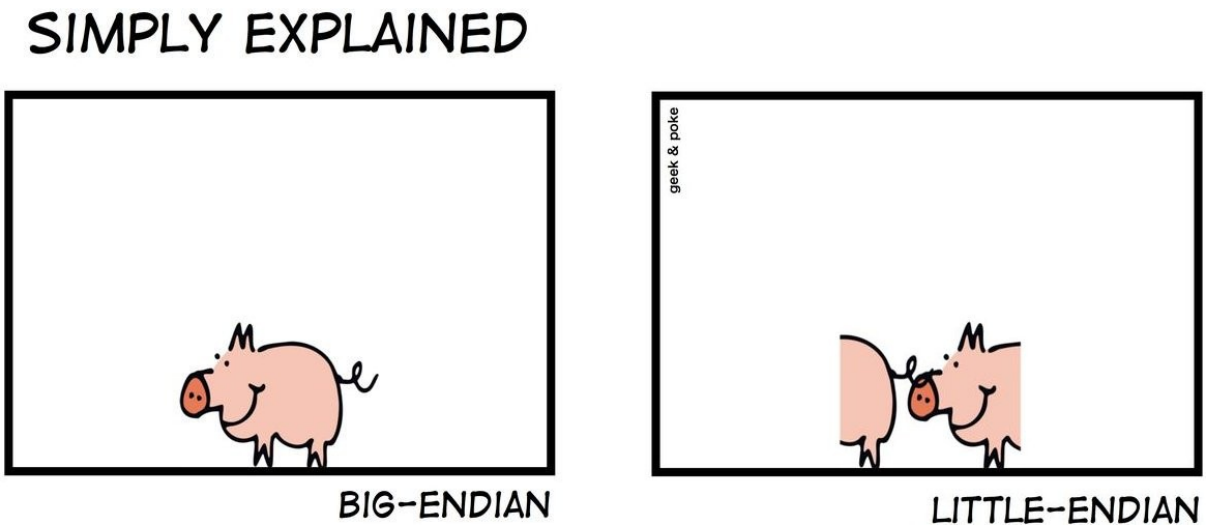
_____

6. Message Queues

Identifikation über Namen, Zugriff über MQ-Deskriptor

_____

Aufgabe 7: (Byteorder, 14 Punkte [3+3+8])



a) Erläutern Sie prägnant in maximal 3 Sätzen den oben abgedruckten Comic (er stammt von der Webseite geekandpoke.typepad.com)

b) Handelt es sich bei dem abgebildeten Tier um ein 8-Bit-, ein 16-Bit- oder ein 32-Bit-Schwein? Erläutern Sie ihre Antwort.

c) Geben Sie ein Assembler-Codefragment an, mit dem Sie testen könnten, ob das Programm auf einer Big-Endian-CPU oder einer Little-Endian-CPU läuft. Hier kommt es uns darauf an, dass Sie eine möglichst effiziente/elegante Lösung mit wenigen Maschineninstruktionen finden. Bitte kommentieren Sie Ihre Lösung ausreichend.

Signaturen relevanter Bibliotheksfunktionen

int **pthread_create**(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine) (void*), void *arg);
int **pthread_attr_init**(pthread_attr_t *attr);
int **pthread_attr_destroy**(pthread_attr_t *attr);
void **pthread_exit**(void *value_ptr);
int **pthread_join**(pthread_t thread, void **value_ptr);
int **pthread_detach**(pthread_t thread);
int **pthread_cancel**(pthread_t thread);
int **pthread_setcancelstate**(int state, int *oldstate);
int **pthread_setcanceltype**(int type, int *oldtype);
void **pthread_testcancel**(void);
int **pthread_mutex_init**(pthread_mutex_t *mutex, pthread_mutexattr_t *attr);
int **pthread_mutex_destroy**(pthread_mutex_t *mutex);
int **pthread_mutexattr_init**(pthread_mutexattr_t *attr);
int **pthread_mutexattr_destroy**(pthread_mutexattr_t *attr);
int **pthread_mutexattr_settype**(pthread_mutexattr_t *attr, int type);
int **pthread_mutexattr_gettype**(const pthread_mutexattr_t *attr, int *type);
int **pthread_mutex_lock**(pthread_mutex_t *mutex);
int **pthread_mutex_trylock**(pthread_mutex_t *mutex);
int **pthread_mutex_unlock**(pthread_mutex_t *mutex);
int **pthread_cond_init**(pthread_cond_t *cond, const pthread_condattr_t *attr);
int **pthread_cond_destroy**(pthread_cond_t *cond);
int **pthread_condattr_init**(pthread_condattr_t *attr);
int **pthread_condattr_destroy**(pthread_condattr_t *attr);
int **pthread_cond_wait**(pthread_cond_t *cond, pthread_mutex_t *mutex);
int **pthread_cond_timedwait**(pthread_cond_t *cond, pthread_mutex_t *mutex, const struct timespec *abstime);
int **pthread_cond_signal**(pthread_cond_t *cond);
int **pthread_cond_broadcast**(pthread_cond_t *cond);
int **sem_init**(sem_t *sem, int pshared, unsigned int value);
int **sem_destroy**(sem_t *sem);
int **sem_wait**(sem_t *sem);
int **sem_trywait**(sem_t *sem);
int **sem_post**(sem_t *sem);
sem_t ***sem_open**(const char *name, int oflag);
sem_t ***sem_open**(const char *name, int oflag, mode_t mode, unsigned int value);
int **sem_close**(sem_t *sem);
int **sem_unlink**(const char *name);
int **pthread_barrier_init**(pthread_barrier_t *restrict barrier, const pthread_barrierattr_t *restrict attr, unsigned count);
int **pthread_barrier_destroy**(pthread_barrier_t *barrier);
int **pthread_barrier_wait**(pthread_barrier_t *);
int **pthread_rwlock_init**(pthread_rwlock_t *rwlock, const pthread_rwlockattr_t *attr);
int **pthread_rwlock_destroy**(pthread_rwlock_t *rwlock);
int **pthread_rwlock_rdlock**(pthread_rwlock_t *rwlock);
int **pthread_rwlock_tryrdlock**(pthread_rwlock_t *rwlock);
int **pthread_rwlock_trywrlock**(pthread_rwlock_t *rwlock);
int **pthread_rwlock_unlock**(pthread_rwlock_t *rwlock);
pid_t **getpid**(void);
pid_t **getppid**(void);

int **system**(const char *command);
pid_t **fork**(void);
int **execl**(const char *path, const char *arg0, ... /*, (char *)0 */);
int **execv**(const char *path, char *const argv[]);
int **execle**(const char *path, const char *arg0, ... /*, (char *)0, char *const envp[]*/);
int **execve**(const char *path, char *const argv[], char *const envp[]);
int **execlp**(const char *file, const char *arg0, ... /*, (char *)0 */);
int **execvp**(const char *file, char *const argv[]);
void **exit**(int status);
void **abort**(void);
pid_t **wait**(int *stat_loc);
pid_t **waitpid**(pid_t pid, int *stat_loc, int options);
int **pipe**(int fd[2]);
int **dup**(int fd);
int **dup2**(int oldfd, int newfd);
FILE ***popen**(const char *cmdstring, const char *type);
int **pclose**(FILE *fp);
int **mkfifo**(const char *pathname, mode_t mode);
mqd_t **mq_open**(const char *name, int oflag);
mqd_t **mq_open**(const char *name, int oflag, mode_t mode, struct mq_attr *attr);
mqd_t **mq_getattr**(mqd_t mqdes, struct mq_attr *attr);
mqd_t **mq_setattr**(mqd_t mqdes, struct mq_attr *newattr, struct mq_attr *oldattr);
int **mq_send**(mqd_t mqdes, const char *msg_ptr, size_t msg_len, unsigned int msg_prio);
ssize_t **mq_receive**(mqd_t mqdes, char *msg_ptr, size_t msg_len, unsigned int *msg_prio);
int **mq_close**(mqd_t mqdes);
int **mq_unlink**(const char *name);
int **mq_notify**(mqd_t mqdes, const struct sigevent *notification);
int **kill**(pid_t pid, int signo);
int **raise**(int signo);
int **sigaction**(int signo, const struct sigaction *act, struct sigaction *oact);
int **sigprocmask**(int how, const sigset_t *set, sigset_t *oset);
int **pthread_sigmask**(int how, const sigset_t *set, sigset_t *oset);
int **sigpending**(sigset_t *set);
int **sigemptyset**(sigset_t *set);
int **sigfillset**(sigset_t *set);
int **sigaddset**(sigset_t *set, int signo);
int **sigdelset**(sigset_t *set, int signo);
int **sigismember**(const sigset_t *set, int signo);
int **pause**(void);
int **sigsuspend**(const sigset_t *sigmask);
int **sigwait**(const sigset_t *set, int *sig);
ssize_t **read**(int filedes, void *buf, size_t nbytes);
ssize_t **write**(int fildes, const void *buf, size_t nbyte);
int **close**(int filedes);
int **fcntl**(int fildes, int cmd, ...);
off_t **lseek**(int filedes, off_t offset, int whence);
int **ioctl**(int fildes, int request, ... /* arg */);
int **socket**(int *domain*, int *type*, int *protocol*);
int **bind**(int *sockfd*, const struct sockaddr **my_addr*, socklen_t *addrlen*);

uint16_t **htons**(uint16_t hvalue);
uint32_t **htonl**(uint32_t hvalue);
uint16_t **ntohs**(uint16_t hvalue);
uint32_t **ntohl**(uint32_t hvalue);
int **sendto**(int sockfd, const void *buff, size_t nbytes, int flags, const struct sockaddr *to, socklen_t tolen);
int select(int maxfdp1, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout);
int **recvfrom**(int sockfd, void *buff, size_t nbytes, int flags, struct sockaddr *from, socklen_t *fromlen);
in_addr_t **inet_addr**(const char *dotted);
char ***inet_ntoa**(struct in_addr network);
int **listen**(int sockfd, int backlog);
int **connect**(int sockfd, const struct sockaddr *serv_addr,socklen_t addrlen);
int **accept**(int sockfd, struct sockaddr *addr,socklen_t *addrlen);
int **close**(int sockfd);
int **shutdown**(int sockfd, int howto);
FD_ZERO (fd_set *set);
FD_SET (int fd, fd_set *set);
FD_CLR (int fd, fd_set *set);
FD_ISSET(int fd, fd_set *set);
int **getsockname**(int socket, struct sockaddr *restrict address, socklen_t *restrict address_len);